

Időpontfoglaló rendszer

Vizsgaremek – Műszaki dokumentáció

Szakirány: Szoftverfejlesztő és -tesztelő technikus

Tanév: 2025/2026

Készítette:

Palotai Péter

Szkibák Szabolcs

Debreceni SZC Mechwart András

Gépipari és Informatikai Technikum

Debrecen, 2026

Tartalomjegyzék

1. Bevezetés, témaválasztás indoklása, kitűzött célok

2. Csapatmunka megvalósítása

2.1 Acsapatmunkaelveieszközei

2.2 Feladatmegosztás

3. Felhasználói dokumentáció

3.1 Rendszerkövetelmények telepítés

3.2 Az alkalmazás indítása

3.3 Bejelentkezés és regisztráció

3.4 Időpontfoglalás lépései

3.5 Saját foglalások kezelése

3.6 Admin felület

4. Fejlesztői dokumentáció

4.1 Fejlesztői környezetek

4.2 Alkalmazott technológiák

4.3 Rendszerarchitektúra – Clean Architecture

4.4 Adatbázis modell

4.5 Backend – ASP.NET Core Web API

4.6 Frontend – webes kliens

4.7 Kiemelt kódrészletek

4.8 Tesztelés és hibajegyzék

5. Összegzés

6. Továbbfejlesztési lehetőségek

7. Irodalomjegyzék

1. Bevezetés, témaválasztás indoklása, kitűzött célok

1.1 Bevezetés

A mindennapi életben egyre nagyobb igény mutatkozik arra, hogy a különféle szolgáltatókhoz – például fodrászatokhoz, kozmetikákhoz, orvosi rendelőkhöz – az időpontfoglalás online, egyszerűen és gyorsan elvégezhető legyen. A hagyományos telefonos egyeztetés helyett a digitális megoldások nemcsak a felhasználók kényelmét növelik, hanem a szolgáltatók adminisztrációs terheit is jelentősen csökkentik.

Vizsgaremekünk egy webalapú időpontfoglaló rendszer teljes megvalósítása, amely modern technológiákat alkalmaz és valós, éles körülmények között is alkalmazható megoldásokat kínál. A projekt a szoftverfejlesztő és -tesztelő technikus vizsgaremek követelményeinek megfelelően, a Debreceni SZC Mechwart András Gépipari és Informatikai Technikum keretein belül készült.

1.2 Témaválasztás indoklása

A témaválasztás során olyan projektet kerestünk, amely:

- Valós üzleti problémát old meg, nem csupán elméleti feladat
- Lehetővé teszi a teljes stack fejlesztés bemutatását: adatbázis, backend API, frontend
- Összetett üzleti logikát tartalmaz: időslot-generálás, ütközésvizsgálat, jogosultságkezelés
- Megfelelő mélységű automatizált tesztelést igényel és lehetővé tesz
- A tanult technológiákat (ASP.NET Core, SQL, JavaScript) komplexen alkalmazza
- Könnyen bővíthető, így a továbbfejlesztési lehetőségek is adóttak

1.3 Kitűzött célok

A projekt tervezésekor az alábbi konkrét, mérhető célokat tűztük ki:

2. Csapatmunka megvalósítása

A projekt kettős fejlesztői csapatban – Palotai Péter és Szkibák Szabolcs közreműködésével – készült. A csapatmunka megszervezésekor tudatosan döntöttünk amellett, hogy nem osztjuk szét mereven a feladatokat rétegek szerint: mindkét fejlesztő aktívan részt vett a projekt összes területén, az adatbázistól a backenden át a frontend kódig. Ez a megközelítés megerősítette a közös felelősségtudatot és a rendszer egészének átlátását.

2.1 A csapatmunka elvei és eszközei

- Közös Git repository (GitHub) – mindkét fejlesztő minden rétegbe commit-olt
- Folyamatos kommunikáció – feladatokat és problémákat közösen vitattuk meg
- Kódnézés (code review) – egymás kódrészeit ellenőriztük és kommentáltuk
- Közös tervezés – adatbázis séma, API végpontok és UI struktúra együtt kialakítva
- Iteratív fejlesztés – kisebb, tesztelhető egységekben haladtunk előre

2.2 Feladatmegosztás

Bár mindkét csapattag hozzájárult minden réteghez, az egyes területeken az alábbi hangsúlyok alakultak ki a fejlesztés során:

Ez a közös fejlesztési megközelítés azt eredményezte, hogy mindkét fejlesztő teljes képet kapott a rendszer működéséről. Ez különösen értékes volt a hibajavítás során, hiszen mindenki tudta, hol kell keresni az esetleges problémákat – legyen az az adatbázis sémában, a service rétegben vagy a frontenden.

3. Felhasználói dokumentáció

3.1 Rendszerkövetelmények és telepítés

Rendszerkövetelmények

Komponens	Minimális követelmény
Operációs rendszer	Windows 10/11, Linux (Ubuntu 20.04+), macOS 12+
.NET SDK	8.0 vagy újabb – dotnet.microsoft.com
MySQL / MariaDB	MySQL 8.0 vagy MariaDB 10.6+
Webböngésző	Chrome 100+, Firefox 100+, Edge 100+
RAM	Minimum 4 GB (fejlesztéshez ajánlott 8 GB)
Live Server (VS Code)	Ritwick Dey bővítmény – opcionális, de ajánlott

Adatbázis létrehozása

1. Indítsd el a MySQL szerveret, majd hozz létre egy új sémát:

```
CREATE DATABASE idopontfoglalo  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

2. Töltsd be a sémát, majd az alapadatokat:

```
mysql -u root -p idopontfoglalo < database/schema.sql  
mysql -u root -p idopontfoglalo < database/seed.sql
```

A seed script létrehozza az alapadatokat: két szerepkört (ADMIN, USER), két demo felhasználót, két helyszínt (Fodrászat, Kosmetika), három szolgáltatást és két dolgozót heti munkaidő-bejegyzésekkel.

Backend konfigurálása

Nyisd meg a **backend/Idopontfoglalo.Api/appsettings.json** fájlt és állítsd be a saját adataid:

```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
      "server=localhost;port=3306;
```

```
        database=idopontfoglalo;  
        user=root;password=SAJAT_JELSZO;  
        TreatTinyAsBoolean=false;"  
    },  
    "Jwt": {  
        "Key": "MINIMUM_32_KARAKTERES_KULCS",  
        "Issuer": "idopontfoglalo",  
        "Audience": "idopontfoglalo"  
    }  
}
```

Fontos: a **TreatTinyAsBoolean=false** beállítás elengedhetetlen, mert a MySQL TINYINT(1) mezőit a Pomelo connector egyébként C# bool-ként értelmezi, ami futási hibát okoz.

3.2 Az alkalmazás indítása

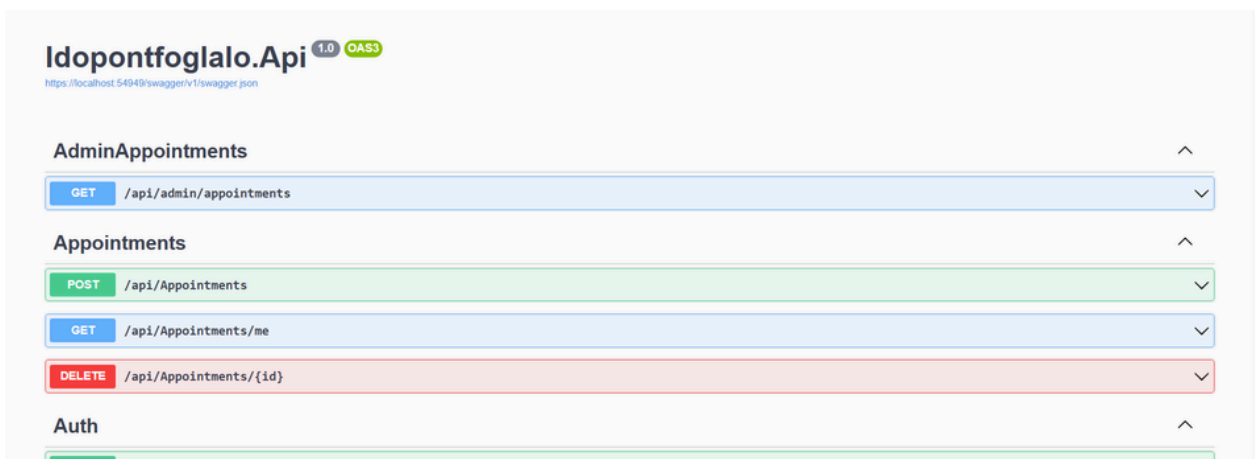
Backend indítása

```
cd backend/Idopontfoglalo.Api
dotnet restore      # NuGet csomagok letöltése
dotnet run
```

Sikeres indítás után:

API: <https://localhost:54949>

Swagger: <https://localhost:54949/swagger>



3.1. ábra – Swagger felület: az API végpontok listája és tesztelési lehetősége

Frontend indítása

A frontend statikus HTML/CSS/JavaScript alkalmazás – nem igényel build lépést. VS Code Live Server bővítménnyel a legegyszerűbb:

- Nyisd meg a **frontend_app** mappát VS Code-ban
- Telepítsd a **Live Server** bővítményt (Ritwick Dey)
- Kattints jobb gombbal az **index.html** fájlra → Open with Live Server
- A böngésző automatikusan megnyílik: **http://127.0.0.1:5500**

A **frontend_app/js/config.js** fájlban ellenőrizd, hogy a baseUrl egyezik a futó backend HTTPS portjával (alapértelmezetten: **https://localhost:54949**).

3.3 Bejelentkezés és regisztráció

Az alkalmazás megnyitásakor a bejelentkezési képernyő jelenik meg. Az alábbi demo fiókokkal lehet azonnal belépni:



3.2. ábra – A bejelentkezési oldal: e-mail, jelszó mezők és a Regisztráció gomb

Regisztráció

Új fiók létrehozásához kattints a "Regisztráció" gombra. Add meg az e-mail címet, jelszót, keresztnévet és vezetéknévet. A rendszer a sikeres regisztráció után azonnal bejelentkezteti az új felhasználót és megjeleníti a foglalási munkaterületet.



3.3. ábra – Regisztrációs nézet az összes kitöltendő mezővel

3.4 Időpontfoglalás lépései

Sikeres bejelentkezés után a foglalási munkaterület jelenik meg. A folyamat négy egyszerű lépésből áll:

1. lépés - Helyszín kiválasztása

A bal oldali kártyán a legördülő menüből kiválasztható a kívánt helyszín (pl. Fodrászat). A kiválasztás hatására a jobb oldali panel automatikusan frissíti a megfelelő dolgozókat és szolgáltatásokat.

3.4. ábra – A foglalási munkaterület helyszín kiválasztó panelje

2. lépés - Szolgáltatás és dolgozó kiválasztása

A jobb oldali foglalási kártyán válaszd ki a kívánt szolgáltatást (pl. Hajvágás – 30 perc, 4 500 Ft) és a dolgozót (pl. Kiss Anna). A rendszer csak azokat a dolgozókat listázza, akik az adott helyszínen dolgoznak és az adott szolgáltatást nyújtják.

3. lépés - Szabad időpontok betöltése

Válassz dátumot a dátumválasztóval, majd kattints a "Szabad időpontok" gombra. A rendszer lekérdezi a backendtől az adott napra vonatkozó szabad slotokat (15 perces lépcsőkkel generálva), és azokat kattintható gombokként jeleníti meg.

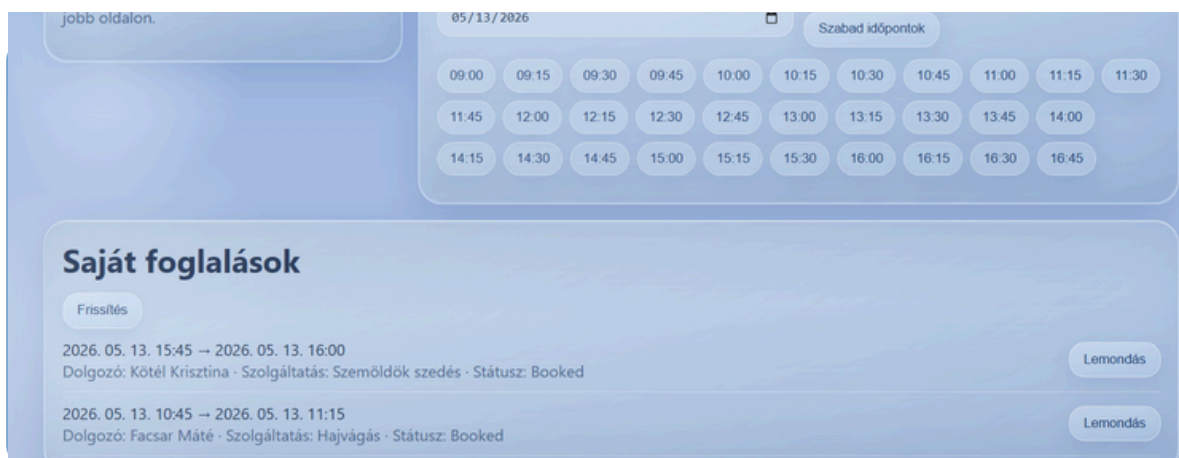
3.5. ábra – Szabad időslotok: kattintható gombokként megjelenítve

4. lépés – Foglalás véglegesítése

Kattints a kívánt időpontra (pl. 10:00). A rendszer HTTP POST kéréssel elvégzi a foglalást. Sikeres esetben megjelenik a "Sikeres foglalás!" visszajelzés, a slot eltűnik a listából, és az időpont bekerül a "Saját foglalások" szekcióba.

3.5 Saját foglalások kezelése

A foglalási munkaterület alatti kártyán a bejelentkezett felhasználó megtekintheti aktív foglalásait. Minden sornál megjelenik az időpont, a dolgozó neve, a szolgáltatás és egy "Lemondás" gomb.



3.6. ábra – Saját foglalások listája lemondási lehetőséggel

A "Lemondás" gombra kattintva a rendszer DELETE kérést küld a backendnek. A foglalás státusza Cancelled-re vált, és az időslot újra elérhetővé válik más felhasználók számára. A lista automatikusan frissül.

3.6 Admin felület

Admin fiókkal bejelentkezve a fejlécben megjelenik az "Admin adminisztrációs modális ablakban az alábbi funkciók érhetők el: felület" gomb. Az

Funkció	Leírás
Foglalások lekérése	Dátumtartomány alapján az összes foglalás listázható
Helyszínek kezelése	Új helyszín felvétele, meglévők szerkesztése, aktiválása
Dolgozók kezelése	Dolgozók felvétele, helyszínhez és szolgáltatáshoz rendelése
Szolgáltatások kezelése	Szolgáltatások CRUD kezelése: név, leírás, időtartam, ár

Admin felület

Csak ADMIN jogosultsággal érhető el.

Tól: 05/01/2026 Ig: 05/15/2026

Foglalások lekérése

2026. 05. 13. 10:45 → 2026. 05. 13. 11:15
Vendég: Palotai Péter · Dolgozó: Facsar Máté · Szolgáltatás: Hajvágás · Státusz: Cancelled

2026. 05. 13. 15:45 → 2026. 05. 13. 16:00
Vendég: Palotai Péter · Dolgozó: Kötél Krisztina · Szolgáltatás: Szemöldök szedés · Státusz: Cancelled

Dolgozók kezelése

Név: E-mail: (opcionális) Telefonszám: (opcionális) Helyszín: Nincs hozzárendelve

☒ Aktív

Nyújtható szolgáltatások

Helyszínek kezelése

Helyszín neve: Aktív ☒

Helyszín mentése Ürlap törlése

Szolgáltatások kezelése

Szolgáltatás neve: Leírás: (opcionális)

Időtartam (perc): Ár (Ft): Aktív ☒

Szolgáltatás mentése Ürlap törlése

Dolgozók listája

Bárdos Márk
E-mail: mbowlh@gmail.com · Telefon: 06705676767 · Helyszín: Fodrászat · Állapot: Aktív
Szolgáltatások: Hajvágás, Szakáll igazítás

Szerkesztés Törés

Szolgáltatások listája

Arc Kezelés
Időtartam: 120 perc · Ár: 8500 Ft · Állapot: Aktív

Szerkesztés Inaktíválás

3.7. ábra – Az admin felület: foglalás-lekérés és entitás-kezelő panelek

4. Fejlesztői dokumentáció

4.1 Fejlesztői környezetek

Eszköz / Környezet	Verzió / Leírás
Visual Studio 2022 Community	Elsődleges IDE – backend C# fejlesztés, NuGet kezelés
Visual Studio Code	Frontend fejlesztés – JavaScript, CSS, HTML szerkesztés
Git + GitHub	Verziókezelés, közös fejlesztés, commit history
.NET SDK 8.0	Backend fordítás és futtatókörnyezet
MySQL 8.0 + MySQL Workbench	Adatbázis-kezelő szerver és vizuális séma tervező
Postman	REST API végpontok manuális tesztelése és debugolása
Live Server (VS Code)	Statikus frontend fejlesztői szerver, hot-reload
xUnit Test Runner (VS2022)	Egységtesztek futtatása és eredmények megjelenítése

4.2 Alkalmazott technológiák

Backend

Technológia / NuGet csomag	Szerepe a projektben
ASP.NET Core 8 Web API	REST API keretrendszer, HTTP végpontok, controller-ek
Entity Framework Core 8	ORM – adatbázis műveletek C# entitásokon keresztül
Pomelo.EntityFrameworkCore.MySql	MySQL/MariaDB provider az EF Core-hoz
Microsoft.AspNetCore.Authentication.JwtBearer	JWT token validálás middleware-ként
BCrypt.Net-Next	Biztonságos bcrypt jelszóhash generálás és ellenőrzés

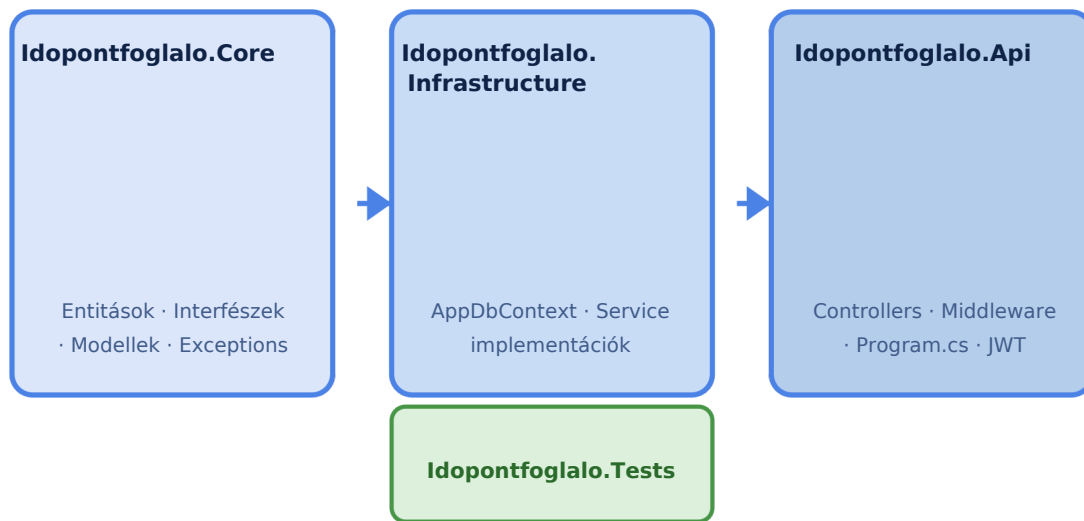
Technológia / NuGet csomag	Szerepe a projektben
System.IdentityModel.Tokens.Jwt	JWT tokenek előállítás (HS256 aláírással)
Swashbuckle.AspNetCore	Automatikus OpenAPI / Swagger dokumentáció generálás
xUnit + Microsoft.Data.Sqlite	Egységtesztelés in-memory SQLite adatbázissal

Frontend

Technológia	Szerepe a projektben
HTML5	Szemantikus dokumentumstruktúra, form elemek
CSS3 – egyedi, keretrendszer nélkül	Glassmorphism stílusú, reszponzív grid layout
Vanilla JavaScript (ES2020+)	Teljes alkalmazáslogika, DOM manipuláció
Fetch API (async/await)	REST API hívások aszinkron kezelése
localStorage	JWT token és szerepkör tárolása munkamenetenként
CSS Custom Properties	Konzisztens szín- és stílusrendszer változókkal

4.3 Rendszerarchitektúra - Clean Architecture

A backend Clean Architecture elvek alapján, három önálló projektre bontva lett megvalósítva. Ez biztosítja a rétegek közötti laza csatolást, az egységes felelősségköröket és a tesztelhetőséget.



4.1. ábra – A backend háromrétegű Clean Architecture felépítése

Idopontfoglalo.Core

Az üzleti logika magja. Tartalmazza az entitásokat (Appointment, Employee, Service, Location, User), az interfészeket (IAppointmentService, IAuthService stb.), a DTO-kat, request/response rekordokat és a BusinessException osztályt. Semmilyen infrastruktúrától nem függ – tisztán C# osztályok.

Idopontfoglalo.Infrastructure

Az adathozzáférési réteg. Tartalmazza az AppDbContext-et (EF Core DbContext), valamint az összes service implementációt (AppointmentService, AuthService, AvailabilityService stb.). Ismeri az adatbázist, de nem ismeri a HTTP réteget.

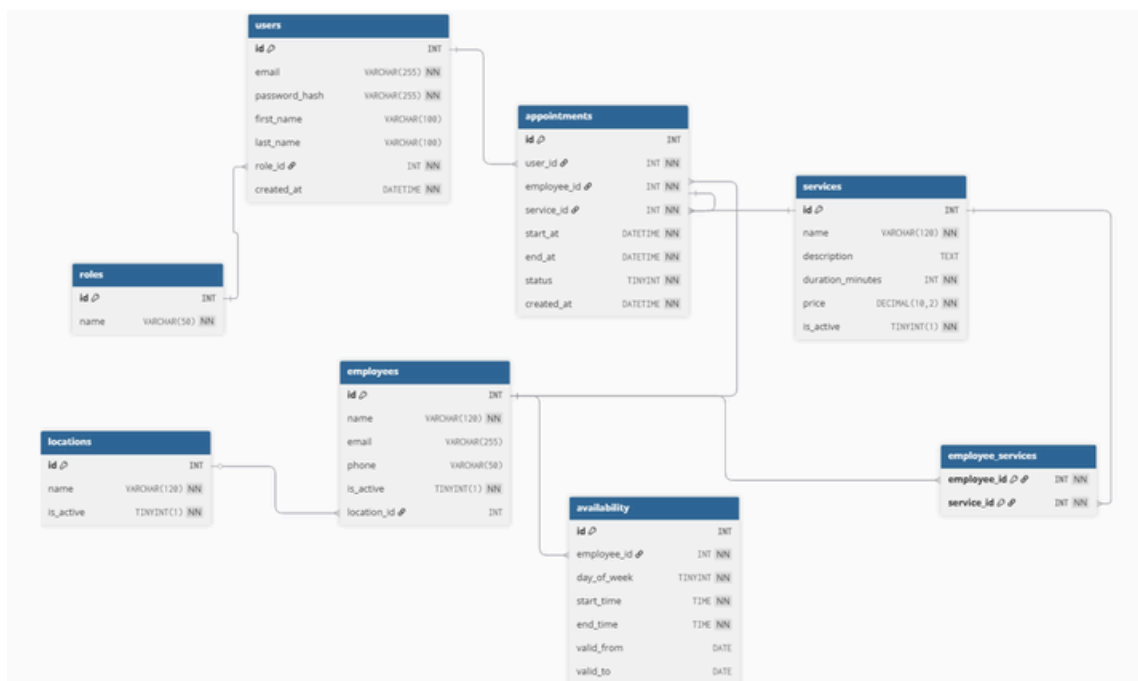
Idopontfoglalo.Api

A prezentációs réteg. ASP.NET Core Web API kontrollerek, globális ExceptionHandlingMiddleware, Swagger konfiguráció, JWT Bearer beállítás és dependency injection (Program.cs). Nem tartalmaz üzleti logikát.

4.4 Adatbázis modell

Az adatbázis 8 InnoDB táblából áll, utf8mb4 karakterkészlettel. összefoglalja a táblákat és azok szerepét.

Az alábbi táblázat



4.2. ábra – Az adatbázis ER-diagramja (a mellékelt adatbazisterv.pdf alapján)

Tábla	Leírás	Kulcsok / Megjegyzés
roles	Felhasználói szerepkörök	id PK, name UNIQUE
users	Rendszerfelhasználók	role_id FK – roles
locations	Foglalási helyszínek	id PK, name UNIQUE
employees	Dolgozók	location_id FK ON DELETE SET NULL
services	Kínált szolgáltatások	duration_minutes, price DECIMAL
employee_services	Dolgozó-szolgáltatás kapcsoló tábla	Kompozit PK (employee_id, service_id)
availability	Heti munkaidő-ablakok	day_of_week 0-6; INDEX (emp, day)
appointments	Foglalások	status: 1=Booked, 2=Cancelled; két index

Tervezési döntések

- **employees.location_id - ON DELETE SET NULL:** helyszín törlése nem törli a dolgozót

- **employee_services, availability - ON DELETE CASCADE:** dolgozó törlése törli a kapcsolódó rekordokat
- **Két index az appointments táblán:** ix_app_employee_start és ix_app_user_start a gyors lekérdezésekhez
- **TreatTinyAsBoolean=false** a connection string-ben: TINYINT(1) mezők helyes C# kezeléséhez

4.5 Backend - ASP.NET Core Web API

API végpontok

JWT autentikáció

A bejelentkezéskor az **AuthService** HS256 aláírású JWT tokenet állít ki. A token 8 órás élettartamú és három claim-et tartalmaz: **NameIdentifier** (felhasználó DB ID), **Email** és **Role** (ADMIN vagy USER). A frontend a tokenet localStorage-ban tárolja, és minden védett kérésnél az **Authorization: Bearer <token>** fejléccel küldi.

ExceptionHandlerMiddleware

A globális kivételkezelő middleware biztosítja, hogy a **BusinessException** típusú hibák HTTP 400 Bad Request-ként, konzisztens JSON formátumban kerüljenek vissza. Váratlan kivételek esetén HTTP 500-at küld és logolja a részleteket.

4.6 Frontend - webes kliens

A frontend egyoldalas alkalmazás (SPA-szerű viselkedés), kizárólag natív webtechnológiákkal (HTML5, CSS3, Vanilla JS). Semmilyen JavaScript keretrendszert, bundlert vagy build eszközt nem igényel.

Fájlstruktúra

```

frontend_app/
├── index.html      # Egyszeres HTML - minden nevet egyszerre jelen
│                  # (.hidden CSS osztállyal mutatva/rejtve)
├── css/
│   └── style.css   # ~450 sor - glassmorphism, reszponziv grid
└── js/
    ├── config.js  # APP_CONFIG.baseUrl - backend URL konfiguráló
    ├── api.js     # apiFetch() helper, fmtDateTime() util
    └── app.js      # ~1120 sor - teljes alkalmazáslogika
  
```

Az app.js fő funkcionális egységei

Funkció / terület	Leírás
Auth modul (onLogin, onRegister, logout)	Bejelentkezés, regisztráció, token mentés, nézet váltás
JWT decoder (decodeJwtPayload)	Token payload base64 dekódolása, role claim kinyerése
Booking panel (loadLocations, loadSlots)	Helyszín / service / employee legördülők, slot gombok
bookSlot(), loadMyAppointments()	Foglalás véglegesítése, saját lista frissítése
Admin modal funkciók	Foglalás-lista, dolgozó / helyszín / szolgáltatás CRUD
setAuthenticatedView()	Panelek megjelenítése / rejtése login állapot alapján
apiFetch() - api.js	Egységes fetch wrapper: auth header, JSON parse, hibakezelés

CSS - Glassmorphism dizájn

- **backdrop-filter: blur() saturate()** – háttér elmosás a kártyákon
- **rgba() + lineáris/radiális gradiensek** – kártyák és fejléc áttetsző háttere
- **CSS Custom Properties** (--glass-fill, --glass-border stb.) – egységes témakezelés

- **CSS Grid + media query** – reszponzív elrendezés 900px és 1000px törésponton
- **box-shadow inset + outer** – mélységérzet és üvegszerű rétegek szimulálása

4.7 Kiemelt kódrészletek

1. Szabad időslot-generálás (AvailabilityService.cs)

A rendszer egyik legösszetettebb üzleti logikája: a munkaidő-ablakok alapján 15 perces lépcsőkkel generálja a lehetséges időslotokat, és minden slotnál ellenőrzi az ütközést a meglévő foglalásokkal (intervallum-metszés algoritmus):

```
var duration = TimeSpan.FromMinutes(service.DurationMinutes);
var step      = TimeSpan.FromMinutes(15); // 15 perces lepcsok

foreach (var av in avs)
{
    var windowStart = date.ToDateTime(
        TimeOnly.FromTimeSpan(av.StartTime));
    var windowEnd = date.ToDateTime(
        TimeOnly.FromTimeSpan(av.EndTime));

    for (var t = windowStart;
         t + duration <= windowEnd;
         t = t.Add(step))
    {
        var slotEnd = t.Add(duration);
        // Utkozsvizsalat: intervallum-metszés
        var overlaps = booked.Any(a =>
            t < a.EndAt && slotEnd > a.StartAt);

        if (!overlaps)
            slots.Add(new SlotDto(t, slotEnd));
    }
}
```

2. JWT token generálás (AuthService.cs)

```
private string CreateJwtToken(User user)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier,
            user.Id.ToString()),
        new Claim(ClaimTypes.Email,
            user.Email),
        new Claim(ClaimTypes.Role,
            user.Role?.Name ?? "USER"),
    };
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(jwtKey));
    var creds = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
```

```
        issuer: issuer, audience: audience,  
        claims: claims,  
        expires: DateTime.UtcNow.AddHours(8),  
        signingCredentials: creds);  
return new JwtSecurityTokenHandler()  
        .WriteToken(token);  
}
```

3. Egységes fetch wrapper – apiFetch() (api.js)

```

async function apiFetch(path, options = {}) {
  const baseUrl = window.APP_CONFIG.baseUrl
    .replace(/\/$/, '');
  const token = localStorage.getItem('token');
  const headers = { ...options.headers };

  if (!headers['Content-Type'] && options.body)
    headers['Content-Type'] = 'application/json';
  if (token)
    headers['Authorization'] = `Bearer ${token}`;

  const res = await fetch(
    baseUrl + path, { ...options, headers });
  const data = await res.json().catch(() => null);

  if (!res.ok) {
    throw new Error(
      data?.error ?? 'HTTP hiba: ' + res.status);
  }
  return data;
}

```

4. Foglalás validáció – négy réteg (AppointmentService.cs)

```

// 1. Aktiv szolgáltatás letezik-e?
var service = await _db.Services
  .FirstOrDefaultAsync(
    s => s.Id == model.ServiceId && s.IsActive)
  ?? throw new BusinessException(
    "A szolgáltatás nem található.");

// 2. Dolgozó nyújtja-e a szolgáltatást?
var provides = await _db.EmployeeServices
  .AnyAsync(es =>
    es.EmployeeId == model.EmployeeId &&
    es.ServiceId == model.ServiceId);
if (!provides) throw new BusinessException(
  "A dolgozó nem nyújtja ezt a szolgáltatást.");

// 3. Nem multbeli időpont?
if (startAt < DateTime.UtcNow.AddMinutes(-1))
  throw new BusinessException(
    "Multbeli időpont nem foglalhato.");

// 4. Az időpont valóban szabad?
var slots = await _availability.GetSlotsAsync(...);
if (!slots.Any(s =>
  s.StartAt == startAt && s.EndAt == endAt))
  throw new BusinessException(
    "Az időpont már nem elérhető.");

```

4.8 Tesztelés és hibajegyzék

Tesztstratégia

A tesztelés xUnit keretrendszerrel, in-memory SQLite adatbázison történik (Idopontfoglalo.Tests projekt). A **TestDbFactory** osztály minden tesztesethez friss, előre feltöltött kontextust hoz létre, így a tesztek egymástól teljesen függetlenül futnak.

Tesztesetek

```
cd backend  
dotnet test
```

```
# Vart kimenet:
```

```
# Passed: 4 | Failed: 0 | Skipped: 0
```

Hibajegyzék

5. Összegzés

A vizsgaremek keretein belül sikeresen megvalósítottunk egy teljes funkcionalitású webalapú időpontfoglaló rendszert. A projekt lefedi a modern szoftverfejlesztés legtöbb fontos területét: relációs adatbázis tervezés és normalizálás, rétegelt backend architektúra (Clean Architecture), RESTful API fejlesztés, biztonságos JWT autentikáció, reszponzív frontend megvalósítás és automatizált egységtesztelés.

A kitűzött célokat teljesítettük: a rendszer képes felhasználók regisztrálására és bejelentkeztetésére, időpontok foglalására és lemondására, szabad időslotok valós idejű generálására ütközésvizsgálattal, valamint admin felületet biztosít a helyszínek, dolgozók és szolgáltatások kezeléséhez.

A csapatmunka hatékonyan valósult meg: Palotai Péter és Szkibák Szabolcs mindketten aktívan részt vettek a projekt összes rétegének kialakításában. A fejlesztés során előkerült hibákat dokumentáltuk és kijavítottuk, az egységtesztek lefedik a kritikus üzleti logikát.

A projekt kiemelkedő technológiai értékei: a Clean Architecture alkalmazása biztosítja a rétegek cserélhetőségét és tesztelhetőségét; a glassmorphism stílusú frontend keretrendszer nélküli megvalósítása demonstrálja a natív webtechnológiák mélységű ismeretét; az ütközésvizsgálatot tartalmazó slot-generáló algoritmus pedig valódi üzleti problémát old meg.

6. Továbbfejlesztési lehetőségek

A jelenlegi rendszer szilárd alapot nyújt számos irányú további fejlesztéshez:

E-mail értesítések és emlékeztetők

Foglalás visszaigazolása és 24 órával előtte emlékeztető küldése – SendGrid vagy SMTP integráción (MailKit könyvtárral) keresztül. Ez a funkció jelentősen csökkentené a meg nem jelenések számát.

Naptárnézet a frontenden

A szabad időpontok vizuális heti naptárban megjeleníthetők (pl. FullCalendar.js), ami áttekinthetőbb és professzionálisabb foglalási élményt nyújt.

Ismétlődő foglaltások

Heti rendszerességgel ismétlődő időpontok egyetlen kattintással lefoglalhatók – új RecurringAppointments tábla és logika szükséges.

Dolgozói önkiszolgáló felület

A dolgozók saját fiókjukon kezelhessék munkaidejüket és lássák foglalásaikat – új EMPLOYEE szerepkör és dedikált frontend nézet szükséges.

Mobilalkalmazás (.NET MAUI)

A meglévő REST API-ra .NET MAUI cross-platform mobilalkalmazás építhető, amely iOS és Android platformokon is elérhető.

OAuth2 / közösségi bejelentkezés

Google, Microsoft vagy Facebook fiókkal való belépés az ASP.NET Core Identity OAuth handler-eivel – csökkenti a regisztrációs súrlódást.

Statisztikai riportok

Adminok számára: foglaltsági hőtérkép, bevételi riport, legnépszerűbb szolgáltatások – chart.js vizualizációval.

Docker konténerizáció

A backend és MySQL Docker Compose-zal konténerizálható – a teljes rendszer egyetlen paranccsal elindítható bármilyen gépen.

Integrációs és E2E tesztek

Az egységtesztek mellé WebApplicationFactory alapú integrációs tesztek, és Playwright / Selenium E2E tesztek a kód megbízhatóságának növelésére.

Több időzóna támogatása

Nemzetközi használathoz DateTime – DateTimeOffset migráció és explicit időzóna-kezelés mind az adatbázisban, mind a frontenden.

7. Irodalomjegyzék

A projekt fejlesztése során az alábbi dokumentációkat, könyveket és online forrásokat vettük igénybe:

- [1]** Microsoft Corporation: ASP.NET Core 8 dokumentáció.
<https://learn.microsoft.com/en-us/aspnet/core/>. (2024)
- [2]** Microsoft Corporation: Entity Framework Core – Getting Started Guide.
<https://learn.microsoft.com/en-us/ef/core/>. (2024)
- [3]** Pomelo Foundation: Pomelo.EntityFrameworkCore.MySql – GitHub repository.
<https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql>. (2024)
- [4]** Auth0 / Okta: JSON Web Tokens – jwt.io bevezető és debugger. <https://jwt.io/introduction/>. (2024)
- [5]** BCrypt.Net: BCrypt.Net-Next – NuGet csomag és dokumentáció.
<https://www.nuget.org/packages/BCrypt.Net-Next/>. (2024)
- [6]** Microsoft Corporation: Swashbuckle és ASP.NET Core – Swagger konfiguráció.
<https://learn.microsoft.com/en-us/aspnet/core/tutorials/getting-started-with-swashbuckle>. (2024)
- [7]** xUnit.net: xUnit.net – Unit testing tool for the .NET platform. <https://xunit.net/>. (2024)
- [8]** Oracle Corporation: MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>. (2024)
- [9]** MDN Web Docs (Mozilla): Fetch API – Web APIs referencia.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API. (2024)
- [10]** MDN Web Docs (Mozilla): CSS backdrop-filter – szűrő effektek.
<https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter>. (2024)
- [11]** MDN Web Docs (Mozilla): CSS Grid Layout referencia.
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout. (2024)
- [12]** Martin, Robert C.: Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, ISBN: 978-0134494166. (2017)
- [13]** Fowler, Martin: Patterns of Enterprise Application Architecture. Addison-Wesley, ISBN: 978-0321127426. (2002)
- [14]** Ritwick Dey: Live Server – VS Code Extension.
<https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>. (2024)
- [15]** Debreceni SZC Mechwart András Gépipari és Informatikai Technikum: Szoftverfejlesztő és -tesztelő képzés leírása. <https://dszcmchwart.hu/kepzesek/>. (2025)